Rumos



Java Backend com Spring Boot

Novembro de 2026

Siga-nos —

C2 General

Agenda da Formação

01. Spring Boot & Rest Básico

02. API Design, DTOs & Validação

03. Persistência com JPA & Hibernate

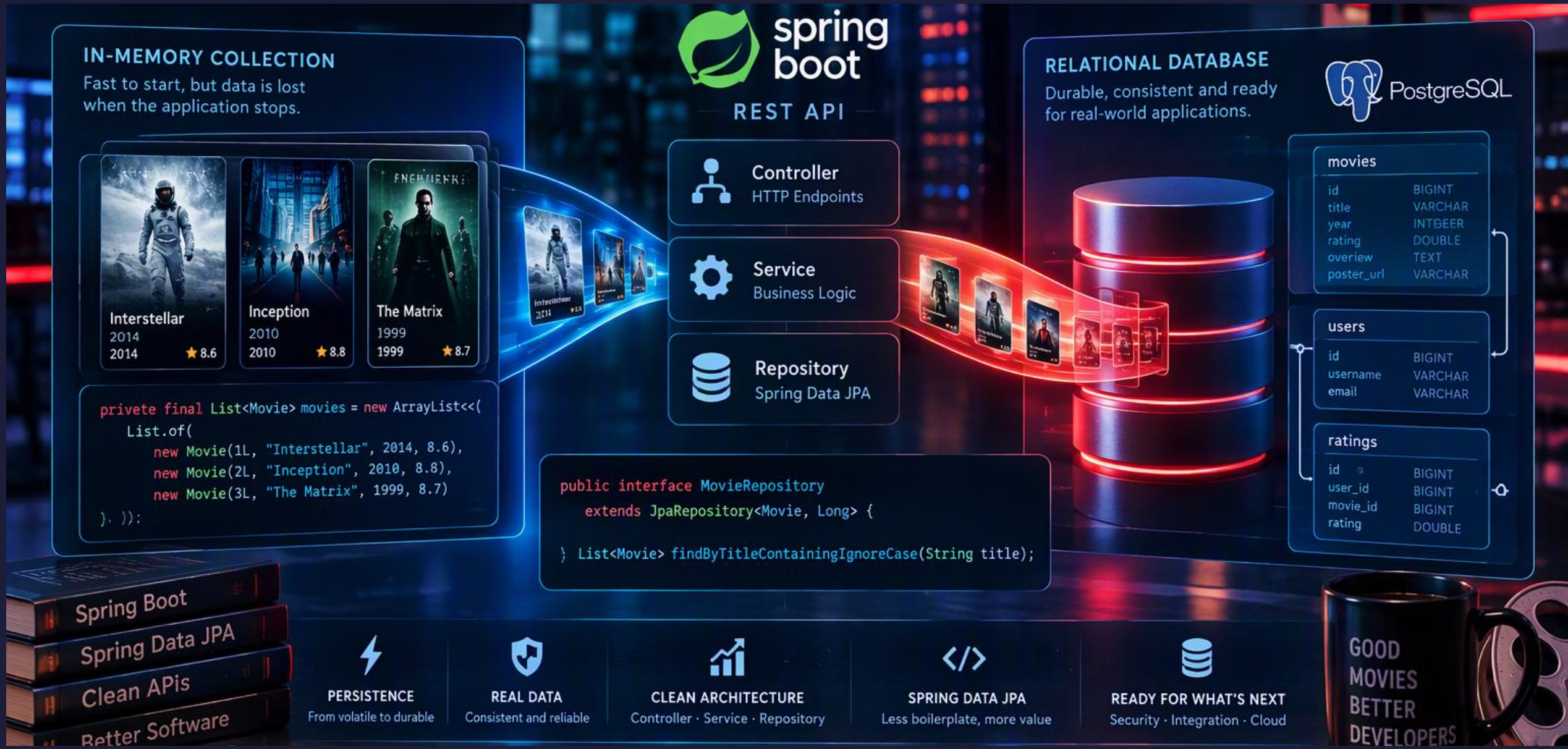
04. Integração Externa (TMDB)

05. Qualidade, Erros & Testes

06. Spring Security, JWT & OAuth2

07. Empacotamento & Container

08. Kubernetes Local (Minikube)


IN-MEMORY COLLECTION
Fast to start, but data is lost when the application stops.
Interstellar
2014
2014 ★8.6
Inception
2010
2010 ★8.8
The Matrix
1999
1999 ★8.7
privete final List<Movie> movies = new ArrayList<<(
List.of(
new Movie(1L, "Interstellar", 2014, 8.6),
new Movie(2L, "Inception", 2010, 8.8),
new Movie(3L, "The Matrix", 1999, 8.7)
).));
spring boot
REST API
Controller
HTTP Endpoints
Service
Business Logic
Repository
Spring Data JPA
public interface MovieRepository
extends JpaRepository<Movie, Long> {
} List<Movie> findByTitleContainingIgnoreCase(String title);
RELATIONAL DATABASE
Durable, consistent and ready for real-world applications.
PostgreSQL
movies
id BIGINT
title VARCHAR
year INTEGER
rating DOUBLE
overiew TEXT
poster_url VARCHAR
users
id BIGINT
username VARCHAR
email VARCHAR
ratings
id BIGINT
user_id BIGINT
movie_id BIGINT
rating DOUBLE
Spring Boot
Spring Data JPA
Clean APIs
Better Software
PERSISTENCE
From volatile to durable
REAL DATA
Consistent and reliable
CLEAN ARCHITECTURE
Controller · Service · Repository
SPRING DATA JPA
Less boilerplate, more value
READY FOR WHAT'S NEXT
Security · Integration · Cloud
GOOD MOVIES BETTER DEVELOPERS

Rumos

Famílias de bases de dados nas aplicações comerciais

Família	Como organiza os dados	Exemplo de utilização
Relacional / SQL	Tabelas, chaves e relações	PostgreSQL, MySQL: encomendas e faturação
Documental	Documentos com campos e listas	MongoDB: catálogos com atributos diferentes
Chave-valor	Uma chave permite obter um valor	Redis: sessões e cache
Colunas largas	Partições e linhas orientadas à consulta	Cassandra: eventos em grande escala
Grafo	Nós ligados por relações	Neo4j: percursos e redes de relações

O modelo deve responder às consultas e às regras do negócio.

Rumos
C2 General

Bases de dados relacionais / SQL

Tabelas, chaves e relações

Primeiro, vamos compreender como os dados se organizam em tabelas.

Depois, vamos mapear as entidades com JPA e configurar o Spring.

Âmbito desta aula

Aprofundaremos bases relacionais/SQL e bases documentais.

Rumos

C2 General

Tabela, linha, coluna e chave primária

Tabela	movies
Linha	um Movie persistido
Coluna	title, director, year, rating
Primary key	id identifica uma linha

Rumos
C2 General

Relações e chaves: um filme pertence a um realizador

directors

id (PK)	name
7	Denis Villeneuve

movies

id (PK)	title	year	director_id (FK)
101	Arrival	2016	7

PK: identifica uma linha.

FK: referencia uma linha válida noutra tabela.

Uma relação 1:N

Um realizador pode ter vários filmes. Cada filme deste exemplo tem um realizador.

Pergunta: o que acontece se guardarmos director_id = 999 sem esse realizador?

Rumos

C2 General

CRUD: quatro operações sobre os dados

Create: criar

```
INSERT INTO movies (title, release_year)
VALUES ('Arrival', 2016);
```

Read: consultar

```
SELECT id, title, release_year
FROM movies WHERE id = 101;
```

Update: alterar

```
UPDATE movies SET title = 'Arrival'
WHERE id = 101;
```

Delete: apagar

```
DELETE FROM movies
WHERE id = 101;
```

O Repository oferece métodos Java que executam operações de persistência.

Rumos

C2 General

Regras, índices e transações

Restrições	NOT NULL, UNIQUE e chaves impedem estados inválidos. Exemplo: um id não se repete.
Índices	Estruturas que podem acelerar pesquisas. Exemplo: procurar filmes pelo ano.
Transações	Uma unidade de trabalho confirma ou reverte alterações.

Rumos
C2 General

Java, Spring, Spring Boot e Spring Data

Java

Classes representam dados e interfaces definem contratos.

Spring

Gere objetos (beans) e fornece dependências pelo construtor.

Spring Boot

Starters e propriedades ajudam a configurar a aplicação.

Spring Data

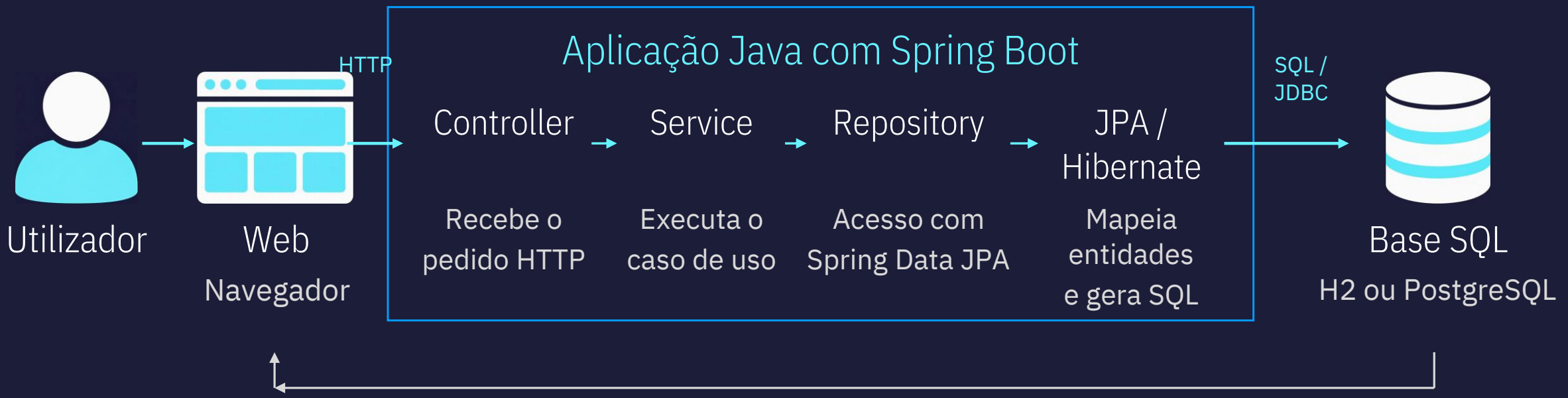
Fornece suporte Repository específico para cada armazenamento.

O projeto inicial já fornece o arranque com @SpringBootApplication.

Rumos
C2 General

Visão geral: do utilizador à base de dados

Exemplo: o utilizador consulta o catálogo de filmes MyMovies.



O resultado regressa pelas camadas. A web apresenta a resposta ao utilizador.

C2 General

O caminho de um objeto Java até ao SQL

Camada	Responsabilidade no exemplo Movie
Service	Executa o caso de uso e delimita a transação
Spring Data JPA	Fornece a implementação de MovieRepository
JPA / Hibernate	Mapeia a entidade para linhas e gera SQL
DataSource / JDBC / driver	Obtém uma ligação e comunica com o SGBD
H2 ou PostgreSQL	Executa SQL e aplica as regras de armazenamento

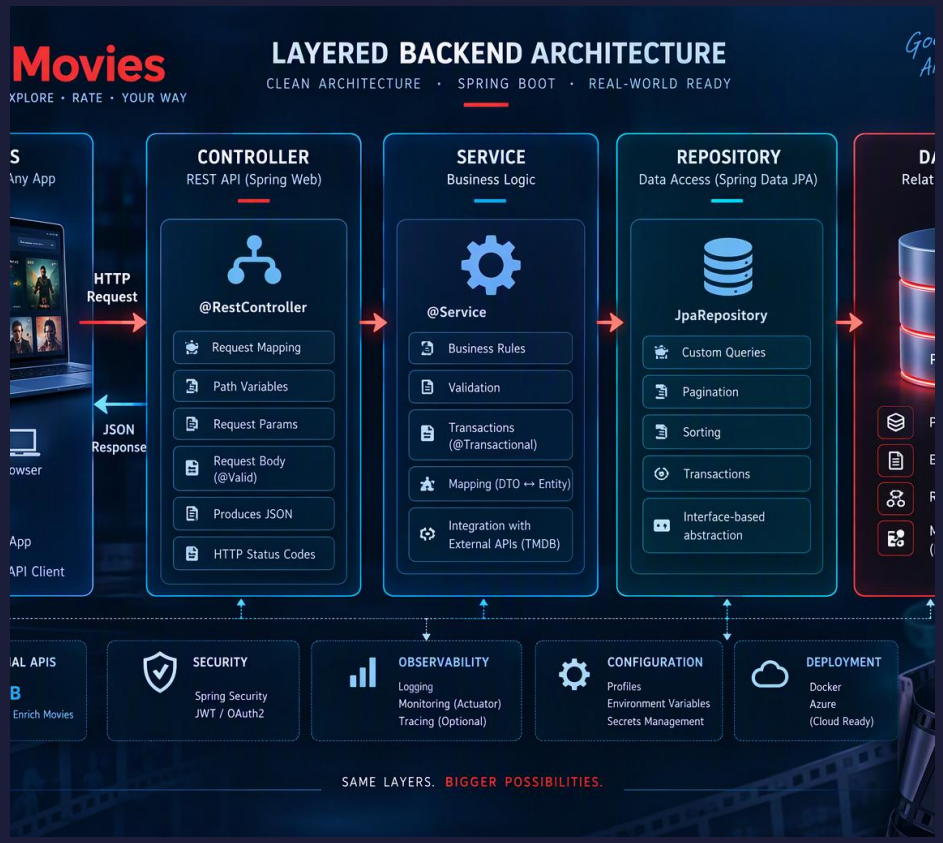
O Spring Boot configura este conjunto a partir das dependências e propriedades.

Rumos
C2 General

Movie: uma classe Java mapeada para uma tabela

```
import jakarta.persistence.*;

@Entity
@Table(name = "movies")
public class Movie {
  @Id
  @GeneratedValue(
      strategy = GenerationType.IDENTITY)
  private Long id;
  @Column(nullable = false)
  private String title;
  @Column(name = "release_year")
  private int year;
  protected Movie() {}
  // construtor publico, getters e setters
}
```



Rumos

C2 General

Como as anotações JPA mudam o mapeamento

Na classe Java	Na base relacional	O que significa
@Entity	Entidade persistente	JPA acompanha este tipo
@Table(name="movies")	Tabela movies	Define o nome da tabela
@Id Long id	Chave primária	Identifica uma linha
@GeneratedValue(IDENTITY)	Valor gerado pela base	Obtém um id ao inserir
@Column(name="release_year")	Coluna release_year	Mapeia o campo Java year
@Column(nullable=false)	Restrição NOT NULL	Impede null se aplicada no esquema

Rumos
C2 General

Spring Data JPA e Repository



C2 General

Estratégia (GenerationType)	Mecanismo de Geração	Vantagens / Atenção	Cenário Recomendado
IDENTITY	Coluna de auto-incremento nativa do SGBD (AUTO_INCREMENT / SERIAL)	• Prós: Muito simples e direto. • Atenção: Desativa o batching de INSERTs do Hibernate.	Labs, APIs simples e SGBDs com auto-incremento (ex.: H2, MySQL)
SEQUENCE	Objeto de Sequência nativo do banco de dados (ex.: PostgreSQL, Oracle)	• Prós: Permite alocação em lote (allocationSize) e inserções em lote. • Mantém alta performance em carga.	Aplicações corporativas de alta performance (PostgreSQL / Oracle)
AUTO	Delega a escolha da estratégia ao Hibernate com base no dialeto do SGBD	• Atenção: Pode alternar entre SEQUENCE, IDENTITY ou TABLE sem controlo explícito.	Dificilmente recomendado para ambiente de produção
TABLE	Simula sequências via tabela dedicada de controlo de IDs no banco de dados	• Prós: Funciona em qualquer SGBD. • Atenção: Pior desempenho (locks e SELECT/UPDATE extra por ID).	Apenas em bases de dados legadas sem suporte a sequências/auto-incremento
UUID	Gera identificador único de 128 bits em memória na aplicação Java 21 (v4 / v7)	• Prós: Geração sem lock no banco, seguro contra varreduras de URL e 100% descentralizado.	Microsserviços, arquiteturas distribuídas e APIs públicas

Quatro relações entre entidades, quatro mapeamentos JPA

Exemplos do catálogo MyMovies. A cardinalidade descreve quantos registos se relacionam.

Relação	Exemplo em Java	Estrutura SQL habitual
Muitos para um @ManyToOne	Movie tem um Director	movies.director_id é uma FK
Um para muitos @OneToMany	Director tem vários Movie	A mesma FK fica em movies
Um para um @OneToOne	Movie tem um MovieDetails	FK em movie_details com UNIQUE
Muitos para muitos @ManyToMany	Movie tem vários Actor Actor participa em vários Movie	Tabela movie_actor com duas FKs

FK = chave estrangeira. Os exemplos acrescentam relações ao modelo básico da aula.

Rumos

C2 General

Muitos filmes para um realizador: @ManyToOne

Em Movie: o lado que guarda a FK

```
@ManyToOne( fetch = FetchType.LAZY,
            optional = false)
@JoinColumn( name = "director_id",
             nullable = false)
private Director director;
```

Na tabela movies

id	title	director_id
1	Filme A	10
2	Filme B	10

Dois filmes referem o mesmo realizador.

Movie controla esta relação.

optional=false exige a relação em Java. nullable=false descreve a coluna NOT NULL.

Rumos

C2 General

Um realizador, vários filmes: @OneToMany e mappedBy

Em Director: a direção inversa

```
@OneToMany(mappedBy = "director")
private List<Movie> movies = new ArrayList<>();

public void addMovie(Movie movie) {
  movies.add(movie);
  movie.setDirector(this);
}
```

Em Movie: o lado proprietário

```
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "director_id")
private Director director;
```

mappedBy="director" referencia o atributo Java em Movie.

A FK fica em movies, mesmo com navegação nos dois sentidos.

Ao alterar a relação, sincronizar os dois lados em Java. O lado proprietário determina a FK.

Rumos
C2 General

Um filme, uma ficha detalhada: @OneToOne

MovieDetails guarda a FK

```
@OneToOne(
    fetch = FetchType.LAZY, optional = false)
@JoinColumn(name = "movie_id",
            nullable = false, unique = true)
private Movie movie;
```

Movie permite navegar para a ficha

```
@OneToOne(mappedBy = "movie",
    cascade = CascadeType.ALL,
    orphanRemoval = true,
    fetch = FetchType.LAZY)
private MovieDetails details;
```

movie_details.movie_id é FK UNIQUE.

Cada filme tem, no máximo, uma ficha.

Neste exemplo, a ficha pertence ao filme. Ao remover a ficha da relação, orphanRemoval permite apagá-la.

A FK fica no lado proprietário. A restrição UNIQUE distingue 1:1 de N:1.

Rumos

C2 General

Filmes e atores: @ManyToMany com tabela de ligação

Em Movie: definição da tabela de ligação

```
@ManyToMany
@JoinTable(name = "movie_actor",
 joinColumns = @JoinColumn(name = "movie_id"),
 inverseJoinColumns =  @JoinColumn(name = actor_id"))
private Set<Actor> actors = new HashSet<>();
```

Tabela de ligação movie_actor

movie_id	actor_id
1	20
1	30
2	20

```
// Direção inversa em Actor
@ManyToMany(mappedBy = "actors")
private Set<Movie> movies = new HashSet<>();
```


Rumos
C2 General

Repository: uma interface para aceder aos filmes

```
1 public interface MovieRepository extends JpaRepository<Movie, Long> {
2   List<Movie> findByTitle(String title);
3 }
```

- findAll / findById
- save / deleteById
- query derivada pelo nome do método

Rumos

C2 General

Métodos básicos já incluídos no Repository

Herdados de JpaRepository<Movie, Long>: o Spring Data gera a implementação ao arrancar.

Método	Devolve	SQL equivalente
findAll()	List<Movie>	SELECT * FROM movies
findById(id)	Optional<Movie>	SELECT * FROM movies WHERE id = ?
save(movie)	Movie	INSERT (id novo) ou UPDATE (id existente)
deleteById(id)	void	DELETE FROM movies WHERE id = ?
count()	long	SELECT COUNT(*) FROM movies

findById devolve Optional porque o filme pode não existir. O SQL é aproximado.

Rumos

C2 General

Convenção: o nome do método descreve a query

Lê-se da esquerda para a direita: findBy + Year + GreaterThan + And + Title + Containing.

Parte	Papel	Exemplos
Prefixo	Define a operação	findBy, existsBy, countBy, deleteBy
Propriedade	Atributo da entidade, em CamelCase	Title, Year, DirectorName
Operador	Condição sobre a propriedade	Containing, Between, GreaterThan
Ligação	Combina várias condições	And, Or
Modificadores	Ordenação, limite, maiúsculas	OrderByYearDesc, Top3, IgnoreCase

Os nomes têm de coincidir com os atributos de Movie; se não existirem, a aplicação falha ao arrancar.

Rumos
C2 General

Exemplos: do nome do método à query executada

Método no MovieRepository	Query gerada (SQL aproximado)
findByYearAndTitle(int y, String t)	WHERE release_year = ? AND title = ?
findByTitleContainingIgnoreCase(String t)	WHERE lower(title) LIKE lower('%' || ? || '%')
findByYearGreaterThanEqual(int y)	WHERE release_year >= ?
findByYearBetween(int a, int b)	WHERE release_year BETWEEN ? AND ?
findByDirectorName(String n)	JOIN directors ... WHERE directors.name = ?
findTop3ByOrderByYearDesc()	ORDER BY release_year DESC LIMIT 3

Os argumentos ligam-se aos ? pela ordem em que aparecem no método.

Rumos
C2 General

@Query: definir uma query personalizada

```java
public interface MovieRepository
                    extends JpaRepository<Movie, Long> {

  @Query("""
      SELECT m FROM Movie m
      WHERE m.year >= :from
      ORDER BY m.title """)
  List<Movie> recentes(@Param("from") int from);


  @Query(value = "SELECT * FROM movies "
      + "WHERE release_year = ?1", nativeQuery = true)
  List<Movie> doAno(int year);
}
```

- JPQL: consulta sobre entidades e atributos Java (Movie, m.year), não sobre tabelas.
- :from liga o parâmetro pelo nome com @Param; ?1 liga pela posição.
- nativeQuery = true usa SQL puro, com tabelas e colunas (release_year).
- A JPQL é validada ao arrancar; o SQL nativo só quando é executado.

Rumos
C2 General

Que abordagem escolher para cada consulta

Começar pela mais simples; passar à seguinte quando o nome do método fica difícil de ler.

Abordagem	Quando usar	Escreve-se em
Query derivada	Filtros simples, com nome legível	Nome do método
JPQL com @Query	Joins, ordenação e filtros mais complexos	Entidades e atributos Java
SQL nativo	Funções ou sintaxe específicas do SGBD	Tabelas e colunas SQL

SQL nativo ata a consulta ao SGBD escolhido; JPQL mantém-se independente dele.

Rumos

C2 General

Service: recebe o Repository pelo construtor

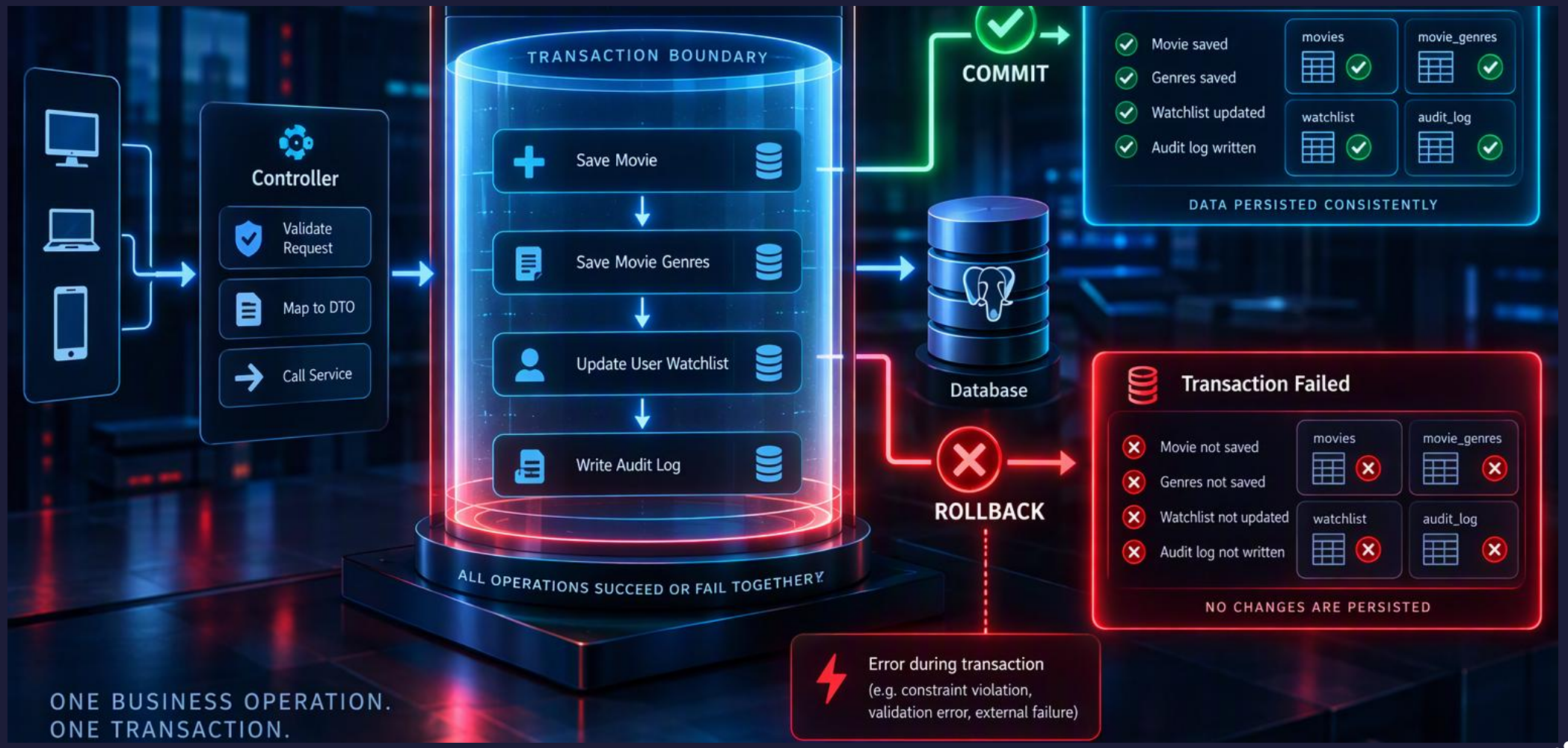
```java
@Service
@Transactional(readOnly = true)
class MovieService {
  private final MovieRepository repository;

  MovieService(MovieRepository repository) {
    this.repository = repository;
  }
}
```

- Controller continua igual conceitualmente.
- Service representa o caso de uso.
- Repository encapsula acesso a dados.
- DTO continua sendo contrato HTTP.

Rumos
C2 General

Fronteira de transação



C2 General

H2 em memória ou em ficheiro: duas configurações

IN-MEMORY

- Muito conveniente para desenvolvimento/testes.
- Dados são descartados quando a aplicação termina.

FILE-BASED NO LAB

- Checkpoint usa arquivo local.
- Permite demonstrar create → restart → read
- Não é a escolha final de produção.

Rumos
C2 General

Configuração SQL do laboratório: JPA e H2

Dependências Maven

```
org.springframework.boot:
  spring-boot-starter-data-jpa
com.h2database:h2  (runtime)
```

application.properties

```
spring.datasource.url=jdbc:h2:file:./data/mymovies
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

Configuração didática. No projeto, o starter parent/BOM gere as versões.

O que o Boot configura

DataSource, JPA/Hibernate e suporte Repository.

O URL escolhe o modo file.

Para comparar: jdbc:h2:mem:mymovies

show-sql permite observar as consultas.

Rumos

C2 General

A mesma aplicação ligada ao PostgreSQL

Driver: org.postgresql:postgresql (runtime)

```
# application-postgres.properties
spring.datasource.url=jdbc:postgresql://localhost:5432/mymovies
spring.datasource.username=${DB_USER}
spring.datasource.password=${DB_PASSWORD}
spring.jpa.hibernate.ddl-auto=validate
```

Antes de iniciar

Criar base e utilizador, aplicar migrações e ativar o perfil postgres.

Guardar credenciais fora do código. Validar o esquema e as permissões.

Rumos
C2 General

Configuração Spring: o suporte JPA é comum às relações

O mesmo starter e as mesmas entidades

spring-boot-starter-data-jpa

Tipos @Entity/@Id nos subpacotes.

```
com.example.mymovies
  Application.java
  domain/     Movie, Director, Actor
  repository/ MovieRepository
  service/    MovieService
```

Propriedades de apoio

```
# Desenvolvimento: observar SQL
spring.jpa.show-sql=true

# Preparar os DTOs no Service
spring.jpa.open-in-view=false

# Produção: esquema já migrado
spring.jpa.hibernate.ddl-auto=validate
```

As migrações criam FKs, UNIQUE e índices. validate verifica o esquema.

As anotações JPA definem as relações. A configuração Boot da ligação mantém-se.

Rumos

C2 General

Laboratório orientado: persistência

Base

1. Mapear Movie com @Entity, @Id e @Column.
2. Configurar H2 em ficheiro e MovieRepository.
3. Injetar o Repository no Service.
4. Criar um filme, reiniciar e consultar o mesmo id.

Extensão

- findByTitleContainingIgnoreCase(String title)

Rumos
C2 General

Bases de dados documentais

Coleções, documentos e campos aninhados

Agora, vamos representar o mesmo filme como um documento.
Mais à frente, vamos mapear e configurar esse modelo
com Spring Data MongoDB.

O exemplo continua a ser o catálogo de filmes MyMovies.

Rumos
C2 General

O mesmo filme como documento

movies: coleção no MongoDB

```
{
  "_id": "movie-101",
  "title": "Arrival",
  "release_year": 2016,
  “rating”: 8.7,
  "genres": ["Sci-Fi", "Drama"],
  "director": {
    "name": "Denis Villeneuve"
  }
}
```

Documento

Uma unidade de dados com campos, listas e objetos aninhados.

Coleção

Agrupa documentos. Os campos podem variar, segundo as regras do modelo.

Rumos
C2 General

Spring Data MongoDB: classe Java e documento

Java

```
@Document(collection = "movies")
public record MovieDocument(
  @Id String id,
  String title,
  @Field("release_year") int year,
  Double rating,
  List<String> genres
) {}
```

Documento BSON, aqui lido como JSON

```
{
  "_id": "movie-101",
  "title": "Arrival",
  "release_year": 2016,
  “rating”: 8.7,
  "genres": ["Sci-Fi", "Drama"]
}
```

record: classe de dados com construtor e acessores gerados pelo Java.

@Document define a coleção. @Id define _id. @Field permite outro nome de campo.

Rumos
C2 General

Beans Spring para o simulador em memória

MongoMemoryConfig.java

```
@Configuration
@Profile("mongo-memory")
class MongoMemoryConfig {
  @Bean(destroyMethod = "shutdown")
  MongoServer mongoServer() {
    return new MongoServer(
        new MemoryBackend());
  }
  // Acrescentar os beans ao lado.
}
```

Restantes beans na mesma classe

```
@Bean(destroyMethod = "close")
MongoClient mongoClient(MongoServer s) {
  return MongoClients.create(
      s.bindAndGetConnectionString());
}

@Bean
MongoTemplate mongoTemplate(MongoClient c)
{
  return new MongoTemplate(c, "mymovies");
}
```

Porta local dinâmica. Beans explícitos ligam o driver e o MongoTemplate ao simulador.

Rumos
C2 General

MongoRepository: um contrato semelhante, outro motor

```
public interface MovieDocumentRepository
                              extends MongoRepository<MovieDocument, String> {

  List<MovieDocument> findByTitleContainingIgnoreCase(String title);
}
```

MovieDocument

O tipo do documento.

String

O tipo do identificador.

Métodos comuns

save, findById, findAll e deleteById.

Rumos

C2 General

Parecido na interface, diferente na persistência

Tema	JPA / SQL	Spring Data MongoDB
Mapeamento	@Entity, @Table, @Column	@Document, @Field
Identidade	jakarta.persistence.Id	org.springframework.data.annotation.Id
Alterar um objeto	Entidade gerida pode sincronizar no commit	Alteração Java exige save ou update explícito
Relações	Tabelas, JOIN e chaves estrangeiras	Embutir dados ou usar referências
Transações	@Transactional com gestor JPA	Atomicidade por documento; várias exigem suporte
Esquema	Migrações e validação	Validação e evolução de documentos

Rumos
C2 General

Checkpoint: bases de dados com Spring

- ✓ Distingo memória, base de dados e SGBD
- ✓ Leio tabelas, chaves e documentos
- ✓ Mapeio Movie e configuro JPA/H2
- ✓ Uso Repository através do Service
- ✓ Provo a persistência após reiniciar
- ✓ Reconheço @Document e MongoRepository

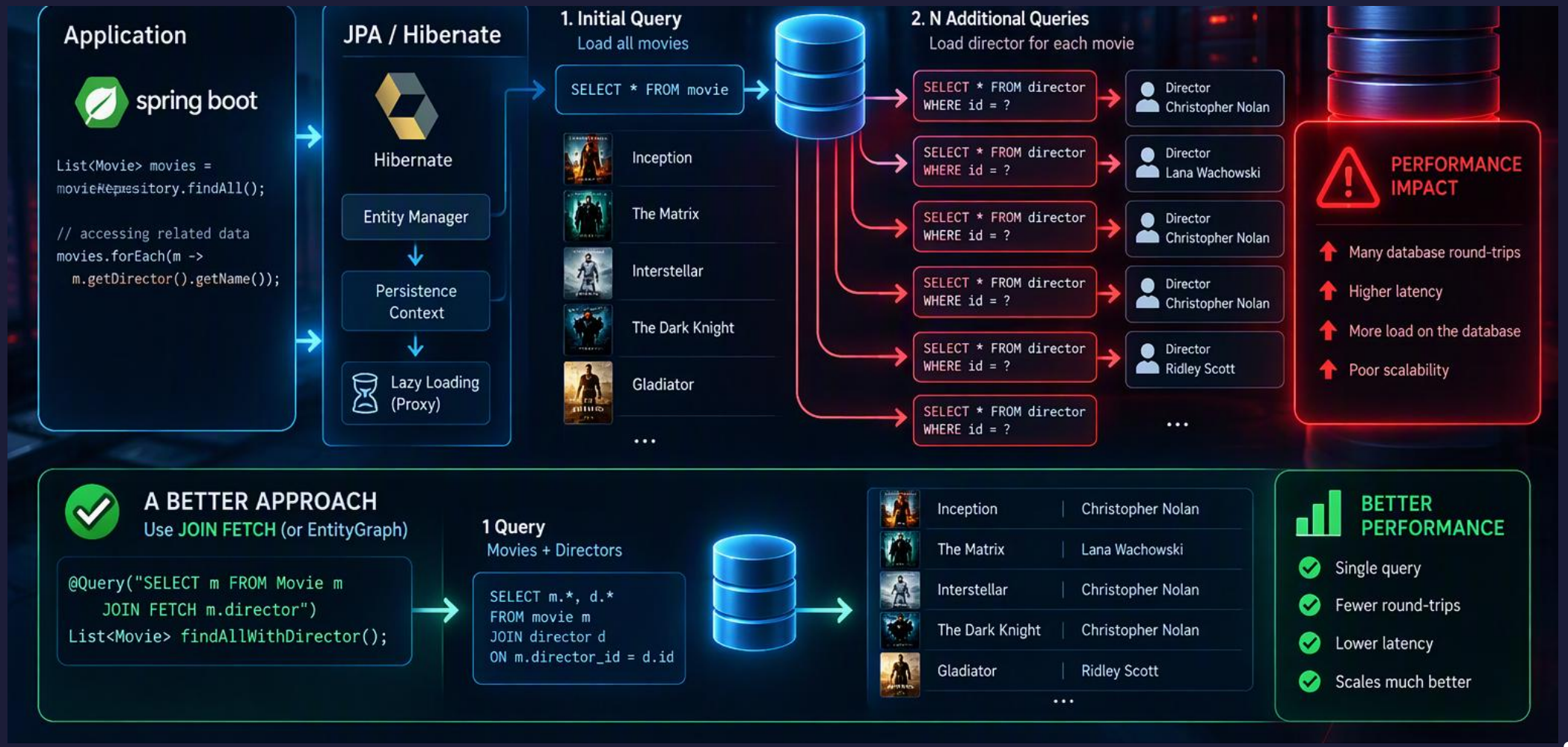
SQL: laboratório
MongoDB: demo guiada

Próximo passo: integrar dados externos no MyMovies, mantendo o contrato HTTP separado da persistência.

Rumos

C2 General

Apêndice: reconhecer o problema N+1



C2 General

Gravar relações: lado proprietário, cascatas e transação

Cascata descreve o ciclo de vida

Opção	Efeito na relação
PERSIST	Propaga a criação/persistência
MERGE	Propaga a cópia de estado
REMOVE	Propaga a remoção
orphanRemoval	Apaga dependentes retirados da relação (1:1 ou 1:N)

Dependentes: MovieDetails ou Review.
Partilhados: Director ou Actor.

Exemplo: realizador já existente

```
@Transactional
public Movie add(Movie movie, Long id) {
  Director director = directors
      .findById(id).orElseThrow();
  director.addMovie(movie);
  return movies.save(movie);
}
```

O helper atualiza os dois lados.

@Transactional agrupa a gravação.

Rumos
C2 General

Carregar relações: LAZY não elimina o N+1

Valores por omissão em JPA 3.x

Relação	fetch por omissão
@ManyToOne	EAGER
@OneToOne	EAGER
@OneToMany	LAZY
@ManyToMany	LAZY

LAZY adia o acesso. EAGER pode gerar consultas adicionais.

```
@EntityGraph(attributePaths = "director")
@Query("select m from Movie m")
List<Movie> findAllWithDirector();
```

Dados necessários na consulta

N filmes e N realizadores distintos podem gerar 1 + N consultas.

Carregar a relação com JOIN FETCH ou @EntityGraph.

Rumos
C2 General